EPSR-test-ex124 Lattice refinement of a cubic box, Bragg peaks with nbraggup = nq

Program(s): epsr | chained passes: 2 | verification: ALL IDENTICAL (all 2 passes)

What this case tests

Exercises the cubic branch of the lattice refinement and of the Bragg-peak calculation of the `epsr` engine [1,2,3], with a Q range of the Bragg calculation that is limited by the number of Q points.

Code exercised

Fortran (original, fortran_src/):

- `epsr_standalone_rev3.f`: `runepsr17` (lattice-refinement loop, cubic branch; Bragg block), `gensq`
- `ato_manipulation_rev4.f`: `bragginten`, `crystfrac` (cubic branch), `change_boxsize`, `atogen`

Rust (port, rust/epsr-core/src/):

- `bragg.rs`: `structure_and_lattice`, `bragginten`, `change_boxsize`, `Bragg::read_keywords`
- `structure.rs`: `atogen`, `gensq`; `rng.rs`: `ran1`

Physical system

A molecular crystal of DMSO and water: a cubic 6.84 Å unit cell with 2 DMSO + 4 water molecules (from `mixato`, `randomise` and 200 Monte Carlo cycles of `epsr` with `potfac 0`), replicated with `makelatticeato` 4 × 4 × 4 into a cubic 27.36 Å box (384 molecules, 2048 atoms, 300 K). The atom types are those of EPSR-test, so its `.pcof`, weights and three D-DMSO/water neutron data files are used unchanged; for the crystal they are numerical input only, not a physical model of measured data. For a crystal the simulation box holds only a few unit cells, so the Bragg part of the structure factor [4] is computed from the reciprocal lattice: intensities per atom pair and reflection are broadened into peaks, divided by the number of unit cells, damped by a Debye-Waller factor and replace the intermolecular partial S(Q) up to `qbraggmax`. Gaussian peaks, no broadening below 0.5 1/Å, Debye-Waller parameter 0.05.

Inputs

`scripts/make_case.sh` regenerates the inputs (verified byte-identical). Keywords changed: `nq 100` (Q up to 5 1/Å), `iinit 1`, `ntimes 2`, `qwidthqmax 0.01 4.5`, `mplicities 4 4 4`, `hklqmin 0.5 5.0 0.05`, `nlatrefine 2`, `latrefine 0.3 0 0 0 0 0`. `commands.txt`: `repeat 2`.

How the accuracy was verified

`commands.txt` (1 program call, repeated 2 times, each pass reading the files written by the previous one) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; only the date line of `.out` and the wall-clock columns of `fort.35` are masked). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in all 2 passes.
`nbraggup` is limited to `nq` ((4.5 + 3/5)/0.05 + 1 = 103 > 100). In the cubic branch one `ran1` call changes a (b = c = a, angles 90°); the trial change is undone exactly by the constant-density rescaling, so the fit does not change and every trial is accepted; on the last step `ilatrefine` is advanced to skip the recomputation. This is the Fortran behaviour and it is reproduced.

References

[1] A. K. Soper, "Empirical potential Monte Carlo simulation of fluid structure", Chem. Phys. 202, 295-306 (1996).
[2] A. K. Soper, "Partial structure factors from disordered materials diffraction data: An approach using empirical potential structure refinement", Phys. Rev. B 72, 104204 (2005).
[3] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[4] G. L. Squires, Introduction to the Theory of Thermal Neutron Scattering, Cambridge University Press (1978).
[5] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.